

ID Test	Classe	Description du Test	Résultat Attendu
TEST-ETU-01	Etudiant	Instanciation par défaut	L'étudiant créé a le prénom "Titouan", le nom "Peloin", l'ID "1" et le cursus "E3e".
TEST-ETU-02	Etudiant	Instanciation avec paramètres	L'étudiant créé prend bien en compte les paramètres (ex: "Maude", "Marius", "2", Cursus E2).
TEST-EPR-01	Epreuve	Instanciation par défaut	Les champs objets sont null et les numériques à 0.
TEST-EPR-02	Epreuve	Instanciation avec paramètres	L'épreuve prend bien en compte les infos (Code, Date, Heure, Durée, Modalité).
TEST-EPR-03	Epreuve	Modification via Setters	Les setters modifient correctement toutes les variables privées de l'épreuve.
TEST-EPR-04	Epreuve	Intégrité de l'Énumération Modalite	L'enum contient exactement 6 valeurs, incluant ORAL, SUR_ORDINATEUR, PROJET et TP.
TEST-FRM-01	Formulaire	Création par défaut	ID généré > 0, date de création et date de modification sont identiques et non nulles. Les listes sont instanciées mais vides.
TEST-FRM-02	Formulaire	Création avec paramètres complets	Le formulaire assigne correctement toutes les listes et informations passées en argument.
TEST-FRM-03	Formulaire	Méthode ajouterEtudiant	L'étudiant est ajouté à la liste, sa taille passe à 1. La date de dernière modification est actualisée (postérieure à la date initiale).
TEST-FRM-04	Formulaire	Méthode ajouterFraude	La fraude est ajouté à la liste. La date de dernière modification est actualisée de la même manière.
TEST-FRD-01	FraudePapier	Constructeurs (Défaut + Paramètres)	Constructeur vide = chaînes vides et non plié (false). Avec params = valeurs assignées et plié (true).
TEST-FRD-02	FraudeCalculatrice	Constructeurs (Défaut + Paramètres)	Constructeur vide = marque "Casio", programme "". Avec params = valeurs assignées (marque, prog, et attributs de la classe mère).
TEST-FRD-03	FraudelAG	Constructeurs (Défaut + Paramètres)	Constructeur vide = nom de service vide "". Avec params = le nom du service est bien stocké.
TEST-FRD-04	FraudelAGConnectee	Constructeurs (Défaut + Paramètres)	Constructeur vide = IP fixée à ""127.0.0.1"". Avec params = IP (ex: ""192.168.0.1"") stockée correctement.
TEST-HST-01	EntreeHistorique	Création d'une entrée	L'action texte est sauvegardée, et l'horodatage est généré automatiquement (non null).
TEST-HST-02	EntreeHistorique	Utilisation des setters	La date et l'action peuvent être écrasées et lues correctement.
TEST-HST-03	JournalHistorique	Création d'un journal	Le journal s'initialise bien avec une liste vide (pas d'erreur NullPointerException).
TEST-HST-04	JournalHistorique	Méthode ajouterEntree	L'ajout de 2 descriptions crée 2 instances EntreeHistorique. Les textes et les dates correspondent.
TEST-HST-05	JournalHistorique	Setter global de la liste	Écraser la liste d'entrées par une nouvelle liste écrase bien l'ancienne.
TEST-HST-06	JournalHistorique	Méthode exporterEnTexte	La fonction génère bien un fichier texte physique sur le disque. Le test supprime le fichier après vérification pour nettoyer.
TEST-HST-07	JournalHistorique	Gestion des erreurs d'export	Si on donne un chemin interdit (ex: . dossier courant), le code gère le try/catch sans faire planter l'application.